RoamMate Sensor API Guide

> **使用原则：** 界面需要持续更新时使用 **StateFlow**；只需读取一次时使用 **Manual Function**；需要响应单次动作时使用 **Event Flow**。

1. 对接入口与基本规则

- **统一入口**：使用应用中共享的 `SensorRepository` 实例，不要在各个 Screen 中重新创建 Repository。
- **调用方式**：普通函数用于单次读取或控制；`StateFlow` 用于持续获取最新状态；`SharedFlow` 用于订阅一次性事件。
- **重要区别**：订阅 Flow **不会自动启动传感器**；相应的传感器需要由应用生命周期管理代码启动，并取得权限。
- **当前实例来源**：`RoamMateApp.sensorRepository` 。`SensorService` 也使用同一个实例。
- **Input = 无** 表示函数无显式参数；`StateFlow` / `SharedFlow` 是可订阅属性，不是可调用函数。

2. SensorRepository 对外 API：Input / Output / Data Type

2.1 Step Counter（累计步数）

API	类型 / 调用方式	Input（类型）	Output（类型）	数据含义 / 注意事项
`stepCountFlow`	状态订阅属性	无；订阅该属性	`StateFlow<Int?>`；内部值 `Int?`	手机**上次重启以来**的累计步数；初始 `null`，停止追踪后可能保留旧值
`getCurrentSteps()`	单次读取函数	无	`Int?`	读取 Repository 缓存的最新累计步数；没有读数时为 `null`
`startStepCounter()`	启动函数（`suspend`）	无	`Boolean`	先从 `StepHistoryStorage` 恢复历史，再尝试启动计步器；`true` 表示启动成功或已经启动；调用可能因数据库操作抛出异常
`stopStepCounter()`	停止函数	无	`Unit`	停止 Step Counter 监听；不保证清除已缓存的步数

2.2 Steps Last Hour（近一小时步数估计）

API	类型 / 调用方式	Input（类型）	Output（类型）	数据含义 / 注意事项
`stepsLastHourFlow`	状态订阅属性	无；订阅该属性	`StateFlow<Int?>`；内部值 `Int?`	最近一次发布的**近一小时估计步数**，不是严格的完整 60 分钟统计；初始 `null`
`getStepsLastHour()`	单次计算函数	无	`Int?`	调用时重新计算近一小时估计；无有效数据时返回 `null`

2.3 Step Detector（单步事件）

API	类型 / 调用方式	Input（类型）	Output（类型）	数据含义 / 注意事项
`stepEvents`	事件订阅属性	无；订阅该属性	`SharedFlow<Long>`；每次事件为 `Long`	每检测到一步发布一个时间戳，**单位毫秒、基于设备启动后的时间基准**；不是 Unix 时间戳，也不是累计步数
`startStepDetection()`	启动函数	无	`Unit`	开始监听单步事件；没有可用硬件时可能不产生事件；不返回启动成功状态
`stopStepDetection()`	停止函数	无	`Unit`	停止单步监听
`isStepDetectorAvailable()`	硬件检查函数	无	`Boolean`	仅检查设备是否有 `TYPE_STEP_DETECTOR`，不代表权限已获批或监听已启动

2.4 GPS Location（位置）

API	类型 / 调用方式	Input（类型）	Output（类型）	数据含义 / 注意事项
`locationFlow`	状态订阅属性	无；订阅该属性	`StateFlow<LocationMessage?>`；内部值 `LocationMessage?`	最新经纬度；初始 `null`，停止定位后可能仍保留上一次位置
`getCurrentLocation()`	单次读取函数	无	`LocationMessage?`	返回 Repository 缓存的最新位置，**不会主动发起一次新的 GPS 定位**
`startLocationTracking()`	启动函数	无	`Unit`	尝试开始定位监听；当前实现要求 `ACCESS_FINE_LOCATION`，不返回是否成功
`stopLocationTracking()`	停止函数	无	`Unit`	停止定位监听；不保证清除缓存位置

2.5 Shake Detector（摇晃事件）

API	类型 / 调用方式	Input（类型）	Output（类型）	数据含义 / 注意事项
`shakeEvents`	事件订阅属性	无；订阅该属性	`SharedFlow<Unit>`；每次事件为 `Unit`	每次检测到摇晃时发出一个**不携带数据**的事件；没有事件时不会发 `false` 或 `null`
`startShakeDetection()`	启动函数	无	`Boolean`	尝试启动加速度计监听；返回是否成功注册（已启动也返回 `true`）
`stopShakeDetection()`	停止函数	无	`Unit`	停止摇晃监听

3. Data Types（所有对接用数据类型）

3.1 自定义 Data Class

`LocationMessage`

```
class LocationMessage(
    val latitude: Double,
    val longitude: Double
)
```

Property	Data Type	含义	示例
`latitude`	`Double`	纬度，十进制度	`-37.810453`
`longitude`	`Double`	经度，十进制度	`144.9631448`

这是当前代码中的普通 `class`，**不是** `data class`；字段均非空。它同时用作 `locationFlow` 的值和 EventBus 的 Payload。

`StepData`

```
data class StepData(
    val steps: Float,
    val timestamp: Long
)
```

Property	Data Type	含义	示例
`steps`	`Float`	记录时手机计步器的累计步数，存储为浮点数	`1245.0f`
`timestamp`	`Long`	记录时的 Unix 时间戳，**毫秒**（`System.currentTimeMillis()`）	`1720000000000L`

`StepData` 用于 `List<StepData>` 历史读取及 `saveStepHistory(record)` 保存。注意：这里的 `timestamp` 与 `stepEvents` 的时间戳**不是同一个时间基准**。

3.2 Kotlin 基础类型与空值规则

Data Type	含义	在本项目中的例子
`Int`	整数	累计步数、估计步数
`Int?`	可空整数	尚未读取到步数时为 `null`
`Long`	64 位整数	单步事件时间戳、历史记录时间戳
`Float`	单精度浮点数	`StepData.steps`
`Double`	双精度浮点数	纬度、经度
`Boolean`	布尔值	`startStepCounter()`、`startShakeDetection()` 的结果
`Unit`	不返回有意义的值	停止函数、摇晃事件的 Payload
`LocationMessage?`	可空位置对象	尚未收到定位时为 `null`
`List<StepData>`	`StepData` 对象列表	数据库读取的步数历史

3.3 Flow / 事件类型

Data Type	代表什么	订阅者收到什么
`StateFlow<Int?>`	持续持有最新可空步数	`Int?`
`StateFlow<LocationMessage?>`	持续持有最新可空位置	`LocationMessage?`
`SharedFlow<Long>`	每次单步事件	`Long`
`SharedFlow<Unit>`	每次摇晃事件	`Unit`（无附加数据）

- `StateFlow` 新订阅者可以立即取得当前缓存值；值可能是 `null` 或旧值。
- 当前 `SharedFlow` 没有配置 `replay`，因此新订阅者**不会收到之前发生的事件**。
- 传感器数据缺失、没有事件、硬件缺失和权限未授予是不同情况，不要直接画等号。

4. 快速调用示例与重要注意事项

使用 App 提供的**共享 `SensorRepository` 实例**。不要在各个 Screen 中重新创建 `SensorRepository`。以下示例假设已获得 `sensorRepository` 引用，且应用已经负责启动对应传感器。

4.1 Manual Functions — 单次读取

需要某个时刻的数据时，主动调用一次函数：

```
val steps = sensorRepository.getCurrentSteps()
val hourlySteps = sensorRepository.getStepsLastHour()
val location = sensorRepository.getCurrentLocation()

val latitude = location?.latitude
val longitude = location?.longitude
```

- `steps`：`Int?`，手机重启以来的累计步数。
- `hourlySteps`：`Int?`，近一小时估计步数；数据不足或不符合当前有效性条件时可能为 `null`。
- `location`：`LocationMessage?`，最新缓存位置；`latitude`、`longitude` 的类型均为 `Double?`。

4.2 StateFlow — 在 Jetpack Compose 中持续观察数据

当传感器发布新状态时，Compose 可以自动更新使用该状态的界面：

```
val steps by sensorRepository.stepCountFlow
    .collectAsStateWithLifecycle()

val hourlySteps by sensorRepository.stepsLastHourFlow
    .collectAsStateWithLifecycle()

val location by sensorRepository.locationFlow
    .collectAsStateWithLifecycle()
```

- `steps`：`Int?`。
- `hourlySteps`：`Int?`。
- `location`：`LocationMessage?`。

4.3 Event Flow — 在 Jetpack Compose 中响应传感器事件

每次检测到单独的步行或摇晃事件时，执行相应逻辑：

```
LaunchedEffect(sensorRepository) {
    sensorRepository.stepEvents.collect { timestamp ->
        // 处理每次检测到的步行事件
    }
}

LaunchedEffect(sensorRepository) {
    sensorRepository.shakeEvents.collect {
        // 处理每次检测到的摇晃事件
    }
}
```

- `stepEvents`：每次事件携带 `Long` 类型的 `timestamp`，单位为设备启动以来的毫秒。
- `shakeEvents`：每次事件携带 `Unit`，表示发生摇晃，不附带额外数据。

4.4 重要注意事项

- **累计步数不是每日步数。** `stepCountFlow` 和 `getCurrentSteps()` 返回手机自上次重启以来的累计计步值。
- **近一小时步数是估算值。** `stepsLastHourFlow` 和 `getStepsLastHour()` 不保证基于完整、连续的 60 分钟历史数据。
- **StateFlow 不会仅因时间流逝而重新计算。** `stepsLastHourFlow` 只有在 Repository 发布新值时才更新。
- **Event Flow 表示事件发生，不是存储的总数。** `stepEvents` 和 `shakeEvents` 适合触发事件处理，不适合直接作为累计步数来源。
- **可用性检查有局限。** `isStepDetectorAvailable()` 只检查硬件；`startStepCounter()` 和 `startShakeDetection()` 返回启动结果 `Boolean`，但普通数据读取 API 没有统一的错误状态码。

- **传感器生命周期由 App 集中管理。** 当前设计中，`MainActivity` 管理前台 Step Detector、GPS 和 Shake 追踪，`SensorService` 管理后台 Step Counter 追踪。其他模块应优先读取或订阅共享 Repository，而不是自行重复启动传感器。